

pfind: A Two-Stage Parallel Directory Traversal Architecture for High-Throughput File Enumeration

Eli Peter (Lehigh University)

Abstract

Directory traversal is a fundamental primitive behind file search, backup, indexing, and developer tooling, yet it often becomes a performance bottleneck at scale due to per-entry syscall overhead, path allocation, and stdout contention when printing results. This paper presents pfind, a Rust prototype that restructures traversal as a two-stage pipeline: (1) a directory-walking stage that enumerates file paths with optional parallel walkers, and (2) a processing stage that consumes paths via a bounded channel and executes user-defined work (metadata, counting predicates, or file reads) in a worker pool. pfind prioritizes a fast baseline hot loop for the common case (walk only, no filtering, no printing), and adds opt-in features through carefully isolated fast paths, including Unix extension filtering before PathBuf allocation and a single-writer printer thread to eliminate stdout lock contention. Across synthetic datasets from 10K to 1M files, pfind achieves the best average execution time among GNU find, fd, os.walk, and bfs, reaching 4.61 million files/sec on a 1M-file flat dataset. At its best configuration (4 walkers, 1 worker), pfind is up to 23.7× faster than GNU find and 2.68× faster than bfs on 1M flat directories, while remaining stable across varied scenarios.

1. Introduction

Fast file enumeration matters. Modern developer workflows repeatedly scan repositories to build indexes, compute dependency graphs, run linters, and search content. System administrators traverse trees to identify large files, enforce policies, and generate inventories. At small scales, directory walking is dominated by fixed overhead and is easily hidden. At large scales (hundreds of thousands to millions of files), however, the traversal itself becomes the limiting factor: each directory entry typically requires syscalls to enumerate and classify entries, and printing results can serialize throughput on stdout.

Traditional tools such as GNU find provide extensive expressiveness and correctness across edge cases, but their general-purpose evaluation model and feature set can impose overhead in pure enumeration workloads. Recent tools such as fd and bfs improve performance through modern implementations and, in bfs, different traversal strategies and parallelism. Python’s os.walk provides portability and convenience but is ultimately limited by interpreter overhead.

This project explores a simple question: What if traversal were explicitly decomposed into two independently tunable stages? pfind separates directory walking from downstream processing. The walker stage focuses on efficiently discovering file paths (optionally in parallel), while the worker stage focuses on doing work per file, buffered by a bounded queue that naturally provides backpressure. This structure enables pfind to (a) retain an extremely fast single-thread baseline, (b) add parallelism where the workload exposes it, and (c) avoid performance pitfalls such as stdout lock contention.

Core contribution: a two-stage traversal architecture where directory discovery and per-file processing are decoupled and independently parallelized/tuned.

2. Compared Systems

pfind is evaluated against common directory-walking or “find-style” tools and baselines:

- GNU find (reference baseline, feature-rich, widely available)
- fd (parallel fast “find alternative”)
- bfs (parallel breadth-first variant of find)
- Python os.walk (standard library baseline)
- Additional walkers, such as basic py_scandir setup and ripgrep’s traversal subsystem, are included as competitors in the throughput/time plots.

The goal is not to match feature sets exactly, but to compare raw traversal throughput under enumeration-like workloads (quiet vs print, all-files vs extension-filtered).

3. Design

3.1 Design goals

- G1: Preserve a fast baseline. When no filtering or printing is requested, pfind keeps the hottest loop minimal: a single stack-based DFS over directories using read_dir and file_type.
- G2: Make parallelism explicit and modular. Directory walking and per-file processing should be separately tunable.
- G3: Minimize synchronization and allocation. Avoid global locks, allocate PathBuf only when required, and buffer output.
- G4: Fail gracefully. When stdout is closed (broken pipe), stop quickly without deadlock.

3.2 Two-stage pipeline

pfind structures execution as a pipeline:

Stage A (Traversal): one or more walker threads enumerate directories and produce file paths.

Stage B (Processing): a pool of worker threads consumes file paths and applies a user function:

- Metadata extraction (process_metadata)
- Predicate counting (walk_and_process_count)
- File reads / checksum (process_with_read)

Stages communicate through a bounded channel (channel::bounded::<PathBuf>(4096) in the prototype). This yields:

- Backpressure: traversal cannot run arbitrarily ahead of processing (bounded memory).
- Bottleneck visibility: if traversal is limiting, workers block on recv; if processing is limiting, walkers block on send.

3.3 Optional parallel traversal (--walkers N)

When enabled, parallel traversal uses:

- A shared lock-free directory queue (SegQueue<PathBuf>)

- A shared pending directory counter (AtomicUsize) tracking the number of scheduled-but-not-yet-finished directories
- Per-walker local directory buffers to reduce contention and improve locality
- A shared stop flag (AtomicBool) to coordinate shutdown (e.g., broken pipe or channel closure)

Termination condition: when the directory queue is empty and pending == 0, walkers stop.

4. Implementation

4.1 Baseline traversal (single walker)

In the default walk-only case (no --print, no --ext, --walkers=1), pfind uses a compact DFS loop:

- Stack of directories (Vec<PathBuf>)
- For each entry: check file_type
- Push directories; increment count for files

This preserves a “hot loop shape” that avoids extra branching and avoids channel overhead when not required.

4.2 Unix extension filtering before PathBuf allocation

When --ext is enabled, pfind performs filtering as early as possible in the traversal stage:

- On Unix: entry.file_name() → raw bytes via OsStrExt::as_bytes()
- Find last dot via memrchr(b'.', name)
- Case-insensitive compare against pre-lowercased extension list

This avoids allocating a PathBuf for non-matching files in the fast-path case.

4.3 Printing without stdout lock contention

Printing is treated as a first-class performance concern.

- In sequential traversal (walk_count_print, walk_and_send_print), a single thread owns the stdout.lock() writer.
- In parallel traversal (--walkers > 1 with --print), each walker accumulates output into 64 KiB byte chunks and sends them to a dedicated printer thread:
 - The printer thread is the only writer to stdout (BufWriter<StdoutLock>)
 - Reduces lock contention and syscall frequency
 - Supports early stop on broken pipe (failed send or failed write_all)

4.4 Worker pool result aggregation

Workers store results in per-thread vectors and merge at the end. This avoids shared synchronization on the result path.

5. Evaluation Methodology

5.1 Hardware and execution environment

All benchmarks were executed locally on a 2023 MacBook Pro (Apple M2 Max) with 12 total CPU cores, running macOS. Experiments were run on the local filesystem to avoid networked storage variability.

Running the full benchmark suite on Sunlab was not practical because the synthetic datasets require substantial on-disk space and fast local I/O to preserve experimental control. The Sunlab environment could not accommodate the dataset storage requirements.

5.2 Synthetic datasets

To isolate traversal behavior and stress different directory topologies, the benchmark suite generates four synthetic datasets:

- Small: 10K files (100 directories $\times$ 100 files/dir), depth $\approx$ 1
- Medium: 100K files (100 directories $\times$ 1,000 files/dir), depth $\approx$ 1
- Large (flat): 1M files (1,000 directories $\times$ 1,000 files/dir), depth $\approx$ 1
- Deep: 1M files (5,000 directories $\times$ 200 files/dir) with maximum depth $\approx$ 10

Datasets are generated programmatically prior to measurement. For each directory, files are created with small contents (e.g., $\sim$ 100 bytes) to avoid “empty-file special cases” while keeping the benchmark dominated by directory enumeration and path handling rather than bulk file I/O.

To support extension-filter workloads, each file is assigned an extension from a fixed set (e.g., jpg, png, txt, json, py, bin) using a weighted distribution (with .jpg and .png more frequent). The generator records the expected file counts per extension so that filtered workloads have a known ground-truth count.

The Deep dataset is constructed to ensure exactly the target number of directories while maintaining a bounded depth: directories are created in a breadth-first manner up to the target depth, then filled out as needed. This avoids pathological “exponential tree” shapes while still stressing traversal in high-directory-count, multi-level layouts.

5.3 Workloads (scenarios)

Each tool is evaluated under four workloads designed to separate traversal cost from output cost, and to test both unfiltered and filtered enumeration:

- All files, quiet: enumerate all files, stdout suppressed
- All files, print: enumerate and write one path per line
- Extension filter, quiet: enumerate only matching files (e.g., .jpg), stdout suppressed
- Extension filter, print: filter and print matching paths

For “quiet” workloads, stdout is redirected to /dev/null (or equivalent). For “print” workloads, output is redirected to a temporary file rather than the interactive terminal, preserving the kernel-level write cost while avoiding terminal rendering overhead.

5.5 Measurement protocol and aggregation

For each (tool, dataset, workload) combination:

- Warmup: 3 warmup runs are executed and discarded (to reduce one-time effects such as cache warming and initial OS/file-system state)
- Measurement: 15 timed runs are executed
- Timing source: wall-clock timing is collected externally (wrapper timing around the subprocess call for all tools), using a high-resolution monotonic timer
- Correctness checks: for each run, the benchmark validates that the number of files enumerated matches the dataset’s expected count (or expected filtered count). Mismatches are flagged

For a fixed tool and dataset, the benchmark first computes the mean runtime for each of the four workloads using the 15 measured runs. Then, figures labeled as “average across all scenarios/workloads” report the average of those workload means (i.e., an equal-weight mean across the four workloads). This ensures that a single workload (like printing) does not implicitly dominate the reported dataset summary unless explicitly analyzed.

When speedups are reported (e.g., vs find or vs the next-fastest parallel tool), they are computed from these workload-averaged runtimes for the corresponding dataset.

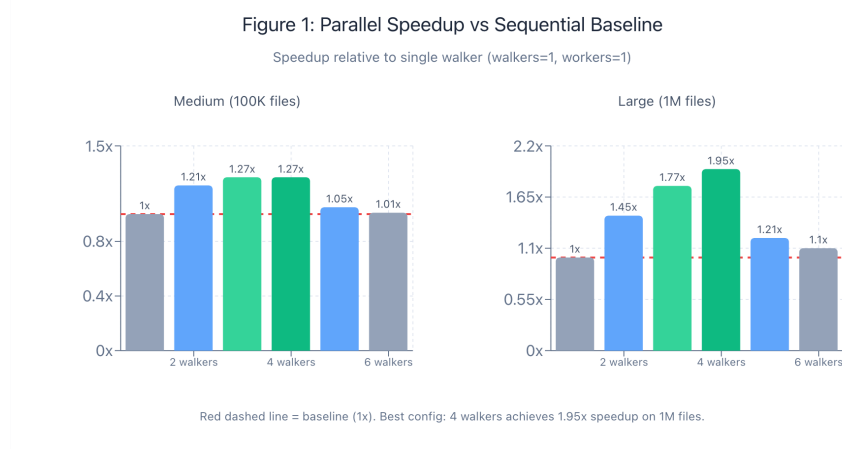
5.6 Reported metrics

Across figures, we report:

- Execution time (lower is better)
- Throughput in files/second (higher is better), computed as `files_enumerated / mean_time`
- Speedup ratios relative to a baseline tool or configuration

6. Results

Figure 1: Parallel speedup vs sequential baseline



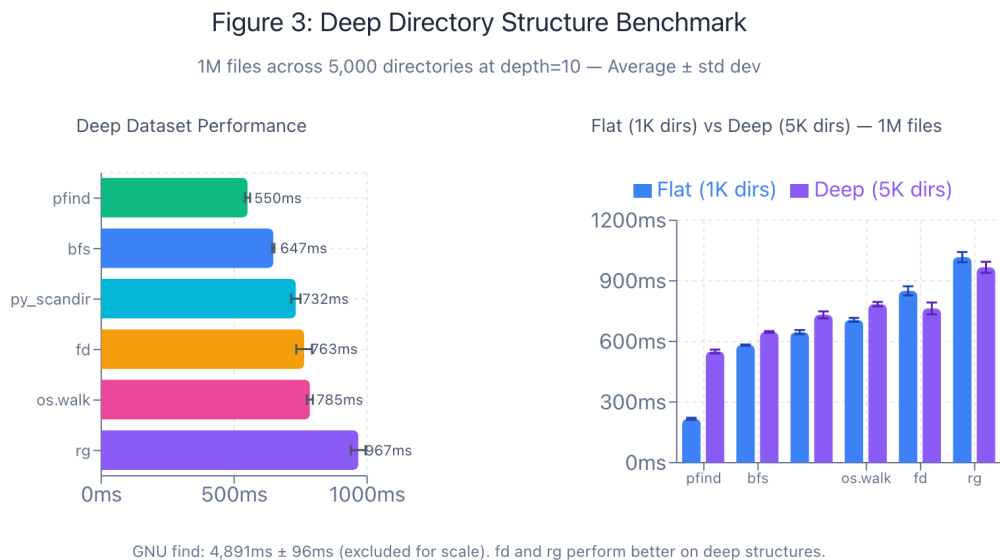
On Medium (100K), pfind reaches $\sim 1.27\times$ speedup around 3–4 walkers. On Large (1M flat), pfind peaks at $\sim 1.95\times$ with 4 walkers, and decreases at 6 walkers, consistent with contention/overhead dominating beyond a small number of traversal threads.

Figure 2: Execution time across tools



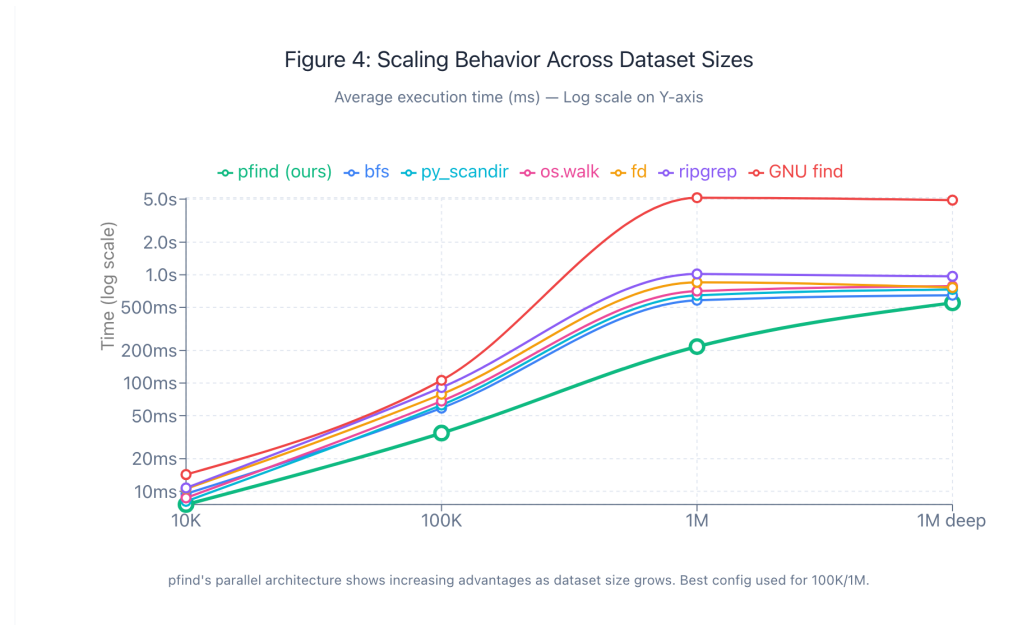
Across 10K, 100K, and 1M datasets (averaged across scenarios), pfind is consistently the lowest-latency tool. On the 1M flat dataset, pfind completes in 217.0 ms, while bfs is 581.7 ms and GNU find is 5153 ms.

Figure 3: Deep directory structure benchmark



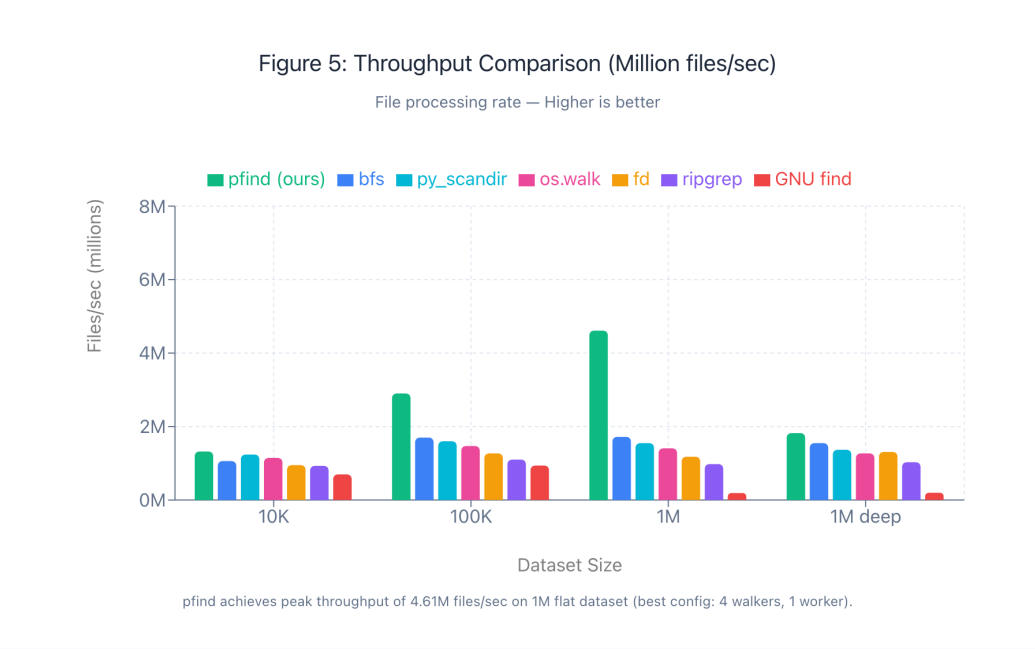
On the 1M deep dataset, pfind remains fastest at 550.3 ms, while bfs achieves 647.2 ms. The gap narrows compared to the flat 1M case, indicating that deeper trees reduce exploitable traversal parallelism and increase queue/coordination overhead.

Figure 4: Scaling behavior across dataset sizes (log scale)



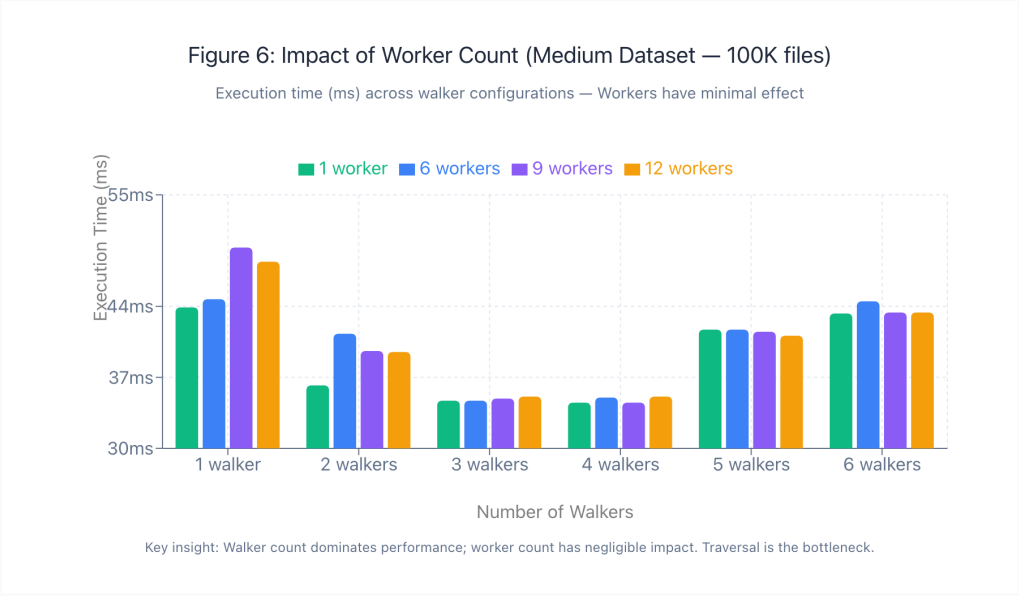
pfind’s advantage grows with dataset size: overhead dominates at 10K, while at 1M the reduced allocation and explicit parallelism have a substantial impact. GNU find exhibits a much steeper growth curve under these enumeration-heavy scenarios.

Figure 5: Throughput (million files/sec)



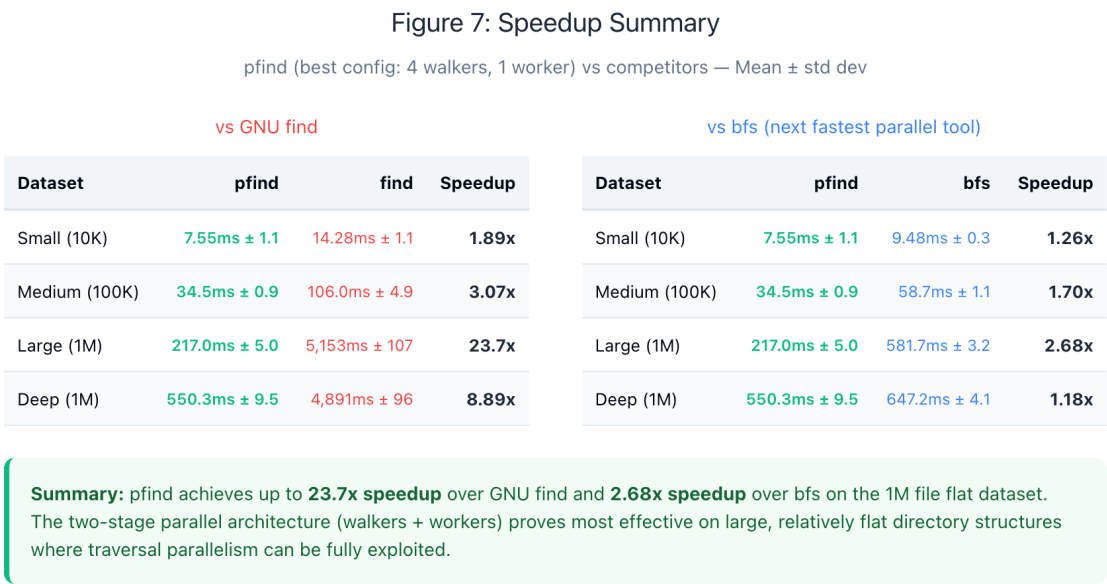
Throughput highlights the main outcome: pfind reaches a peak of 4.61M files/sec on the 1M flat dataset (best config: 4 walkers, 1 worker). Throughput decreases on the deep dataset, reinforcing that directory structure (frontier width and depth) matters as much as file count.

Figure 6: Impact of worker count (100K dataset)



Varying worker count from 1 to 12 yields minimal improvement (and sometimes slight regression), while walker count dominates performance. This supports the central thesis: traversal is the bottleneck in walk-heavy workloads, so adding processing threads does not help unless the processor itself is expensive.

Figure 7: Speedup summary vs GNU find and bfs



On 1M flat, pfind achieves 23.7 $\times$ speedup over GNU find and 2.68 $\times$ over bfs (averaged across scenarios). On 1M deep, pfind retains an advantage but smaller: 8.89 $\times$ over find and 1.18 $\times$ over bfs.

7. Discussion

7.1 Traversal dominates

Figures 1 and 6 jointly demonstrate that performance is primarily governed by the traversal stage: increasing walker threads yields measurable gains up to a point, while increasing worker threads yields little benefit for traversal-heavy workloads.

7.2 Why two stages help

The two-stage architecture makes performance tuning straightforward:

- If traversal is limiting: adjust walker parallelism, reduce allocations, improve directory-queue contention behavior.
- If processing is limiting: increase worker count, adjust queue size, batch processing work.

Importantly, the bounded channel provides a simple, robust form of backpressure without complex flow control logic.

7.3 Deep trees reduce parallel opportunity

Deep directory structures constrain the available parallel frontier. Walkers spend more time discovering the next wave of directories, reducing the steady-state supply of independent work. Shared-queue coordination overhead also becomes more visible relative to useful traversal work.

7.4 Output is a workload

Printing millions of paths can easily dominate wall time. `pfind`'s single-writer printer thread prevents lock contention and reduces syscall frequency via chunking, but output bandwidth remains a fundamental limit when `--print` is enabled.

8. Limitations and Future Work

This evaluation uses synthetic trees. Real-world filesystems involve symlinks, permission errors, ignore rules, cold-cache effects, and heterogeneous storage. Competitors also support richer features that were not the focus of this work.

Future improvements include:

- Lower-contention traversal scheduling (e.g., work-stealing dequeues for directories rather than a single shared queue).
- Adaptive tuning (auto-select walker count based on observed contention and filesystem behavior).
- Ignore rules / globbing is integrated without compromising the baseline hot loop.
- Platform-specific batching (where available) for directory reads and metadata operations.

9. Conclusion

`pfind` demonstrates that explicitly splitting directory traversal into a directory-walking stage and a processing stage yields a simple, high-throughput architecture. By preserving a minimal baseline hot loop and selectively adding parallelism and buffering where it matters (parallel

walkers, bounded queues, and single-writer printing), pfind achieves state-of-the-art throughput on large flat datasets and remains competitive on deep directory structures. The results emphasize a practical systems lesson: for many enumeration workloads, optimizing traversal itself is more impactful than scaling downstream processing.

References

GNU findutils. "find" command documentation and manual pages (findutils).

sharkdp. "fd" - A simple, fast and user-friendly alternative to find (project documentation).

BurntSushi. "ripgrep" - recursively searches directories for a regex pattern (project documentation).

T. Barnes. "bfs" - breadth-first search for your files; includes asynchronous, parallel directory traversal (project documentation).

Python Software Foundation. Python "os.walk" and "os.scandir" documentation; PEP 471 (os.scandir) for performance motivation.

Crossbeam Project. crossbeam-channel and crossbeam-queue documentation (bounded channels and lock-free queues).

D. Lemire et al. "memchr" crate documentation (fast byte search routines).